

Interfaçage API REST et Format JSON

Objectif

L'objectif de cet atelier est de manipuler le format universel d'échange de données du Cloud (le JSON) et de faire communiquer vos scripts PowerShell avec des services web distants via des requêtes HTTP (REST).

Vous utiliserez une API publique gratuite spécialement conçue pour les tests de développement : JSONPlaceholder.

Exercice 1 : Manipulation Locale du Format JSON (Sérialisation)

Avant de parler au réseau, vous devez savoir formater vos données.

1. Création de la donnée source :

Déclarez une table de hachage nommée `$MonServeur` représentant la configuration d'une machine. Elle doit contenir :

- Nom : "SRV-DOCKER-01"
- OS : "Ubuntu 22.04"
- EnProduction : `$true`
- Services : Un tableau contenant "Nginx", "Docker", "Prometheus"

2. Conversion (Sérialisation) :

Utilisez la commande `ConvertTo-Json` pour transformer votre table de hachage en chaîne de caractères JSON et affichez le résultat à l'écran.

3. Le piège de la profondeur :

Modifiez votre variable `$MonServeur` pour ajouter une clé Réseau. Cette clé doit elle-même contenir une table de hachage avec les clés IP ("10.0.0.5") et Passerelle ("10.0.0.1").

Relancez `ConvertTo-Json`. Que remarquez-vous concernant l'affichage de la clé Réseau ?

4. La remédiation :

Ajoutez le paramètre `-Depth 3` (ou supérieur) à votre commande `ConvertTo-Json` pour forcer PowerShell à lire l'arborescence complète et validez que l'IP s'affiche correctement dans le JSON.

Exercice 2 : Lecture de Données via une API REST (Méthode GET)

Nous allons interroger l'annuaire fictif de notre API de test.

1. La requête brute :

Utilisez `Invoke-RestMethod` pour envoyer une requête GET à l'URL suivante :

<https://jsonplaceholder.typicode.com/users>

Stockez le résultat dans une variable \$UtilisateursAPI.

2. Validation du type :

Faites un Get-Member sur \$UtilisateursAPI.

Constat : Bien que l'API renvoie du texte JSON, PowerShell l'a **automatiquement désérialisé**. Vous avez entre les mains de véritables objets personnalisés ([PSCustomObject]).

3. Traitement des données :

Puisque ce sont des objets PowerShell standards, envoyez \$UtilisateursAPI dans le pipeline.

Utilisez Select-Object pour n'afficher que le nom de l'utilisateur (name), son email (email), et la ville où il habite (address.city).

Astuce : Vous devrez utiliser une propriété calculée (table de hachage) pour la ville.

Exercice 3 : Création d'une Ressource sur l'API (Méthode POST)

Vous allez simuler la création d'un "Ticket" ou d'un "Article" sur le serveur distant. L'API JSONPlaceholder accepte les créations fictives (elle vous renverra un ID de confirmation, sans modifier sa base de données réelle).

1. Préparation du Payload (Le corps de la requête) :

Créez un objet ou une table de hachage PowerShell (\$NouveauTicket) contenant 3 propriétés :

- title : "Alerte Supervision"
- body : "Le CPU du serveur de base de données a dépassé 90%."
- userId : 101

2. L'envoi vers l'API :

Utilisez Invoke-RestMethod pour envoyer ces données vers l'URL :

<https://jsonplaceholder.typicode.com/posts>

Contraintes obligatoires pour la commande :

- La méthode doit être POST.
- Le corps de la requête (-Body) doit être votre variable **convertie en JSON**.
- Le type de contenu (-ContentType) doit être déclaré comme "application/json".

3. Validation :

L'API doit vous répondre en vous renvoyant votre objet accompagné d'une nouvelle propriété id (généralement 101). Affichez cette réponse pour confirmer le succès de l'opération.

Exercice 4 : Gestion de la Résilience (Le Try/Catch sur HTTP)

En automatisation CI/CD, un appel d'API qui échoue (serveur injoignable, ressource introuvable)

doit être intercepté proprement, sinon le script plante.

1. Le test d'échec :

Faites un `Invoke-RestMethod (GET)` vers une URL volontairement fausse :

`https://jsonplaceholder.typicode.com/ressource-fantome-404`

Constatez l'erreur rouge bloquante (Erreur 404).

2. La sécurisation du script :

Encapsulez cette requête fautive dans un bloc `try { ... } catch { ... }`.

N'oubliez pas d'ajouter `-ErrorAction Stop` à votre commande `Invoke-RestMethod`, sinon le bloc `Catch` ne s'activera pas !

3. L'affichage propre :

Dans le bloc `catch`, affichez un message d'alerte en jaune expliquant que l'API est injoignable, et extrayez le message d'erreur HTTP exact via `$_Exception.Message`.